

Understanding our address spaces / CIDRs

- **Virtual network / subnet address spaces**
 - Each AKS cluster will have a **Virtual Network** (logical network used to connect resources), and the AKS resources, like nodes, will be part of a **subnet**, which can use the entire address space of a Virtual Network or only a part of it
- **Pod CIDR**
 - Represents the address range from where the pods will assign the IP address
 - This address range should not be used in any other network environment that AKS uses or connects to, like the AKS subnet, range(s) from the AKS VNET that the cluster may connect to, on-premises, Azure virtual networks using Express Route or a Site-to-Site VPN connection, and others
 - This cannot be changed after cluster creation, so a large range is recommended
 - Same range can be reused by other AKS clusters
 - NOTE!: It is possible for the pods to have IPs from the subnet; we will understand how later

Understanding our address spaces / CIDRs

- **Service CIDR**

- Represents the address range that internal services will assign the IP addresses from, like the IP addresses assigned to the ClusterIP
- This address range should not be used in any other network environment that AKS uses or connects to, like the AKS subnet, range(s) from the AKS VNET that the cluster may connect to, on-premises, Azure virtual networks using Express Route or a Site-to-Site VPN connection, and others
- This cannot be changed after cluster creation, so a large range is recommended
- Same range can be reused by other AKS clusters
- In this CIDR, there is also the **Kubernetes DNS service IP address** used by cluster service discovery. In AKS, by default, that is the service exposing CoreDNS pods

```
$kubectl get svc kube-dns -n kube-system
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
kube-dns  ClusterIP   10.0.0.10     <none>         53/UDP,53/TCP  14h
$kubectl describe endpoints kube-dns -n kube-system | grep "Addresses:"
Addresses:      10.244.0.4,10.244.0.8
NotReadyAddresses: <none>
$kubectl get pod -n kube-system -o wide | grep "coredns-"
coredns-59b6bf8b4f-5cwt8      1/1      Running    0           14h      10.244.0.8      aks-nodepool11-35333449-vmss
000000      <none>    <none>
coredns-59b6bf8b4f-pb6ph      1/1      Running    0           14h      10.244.0.4      aks-nodepool11-35333449-vmss
000000      <none>    <none>
```